Student Name: Pitchaimani N | **Student ID:** 2025AE05341 | **Course No:** AIMLZG523 | **Course Title:** MLOps - Assignment 1

Heart Disease Prediction MLOps Project Report

1. Project summary

This project demonstrates an end-to-end machine-learning workflow for the Cleveland Heart Disease dataset: data download and cleaning, exploratory data analysis, model comparison and MLflow tracking, a validated FastAPI endpoint, Docker packaging, Kubernetes deployment, and GitHub Actions automation.

The classifier is for coursework and demonstration only. It is not a medical device and its outputs must not be used for diagnosis, treatment, or clinical decision-making.

2. Workflow and components

1. `download.py` downloads the UCI source file, audits missing values, maps the original outcome to a binary target, and writes the prepared CSV.
2. `notebooks/eda.ipynb` inspects the prepared data and produces labeled feature histograms, a correlation heatmap, and a target class-balance chart.
3. `train.py` compares Logistic Regression and Random Forest pipelines, logs experiments to MLflow, and saves the selected fitted pipeline.
4. `api/main.py` serves `/health` and `/predict`; `tests/test_app.py` checks successful requests and input validation.
5. The Docker image packages the API and trained model. Kubernetes runs two API pods behind a Service.
6. `.github/workflows/main.yml` repeats download, training, lint, and tests on GitHub Actions and uploads generated training outputs.

3. Dataset preparation and EDA

The cleaned Cleveland dataset contains 303 records, 13 predictor columns, and one binary target. Target 0 indicates no disease in the source label mapping; target 1 groups original Cleveland outcome values 1-4 as disease. The observed class counts are:

Target	Meaning	Records	Share
0	No heart disease	164	54.13%
1	Heart disease	139	45.87%

There are six missing predictor values in the source: four in `ca` and two in `thal`. They are retained in the cleaned data and handled by median imputation inside

each model pipeline. The class proportions are reasonably close, and the training code uses stratified splitting and cross-validation to preserve the class mix.

The EDA notebook saves the following figures:

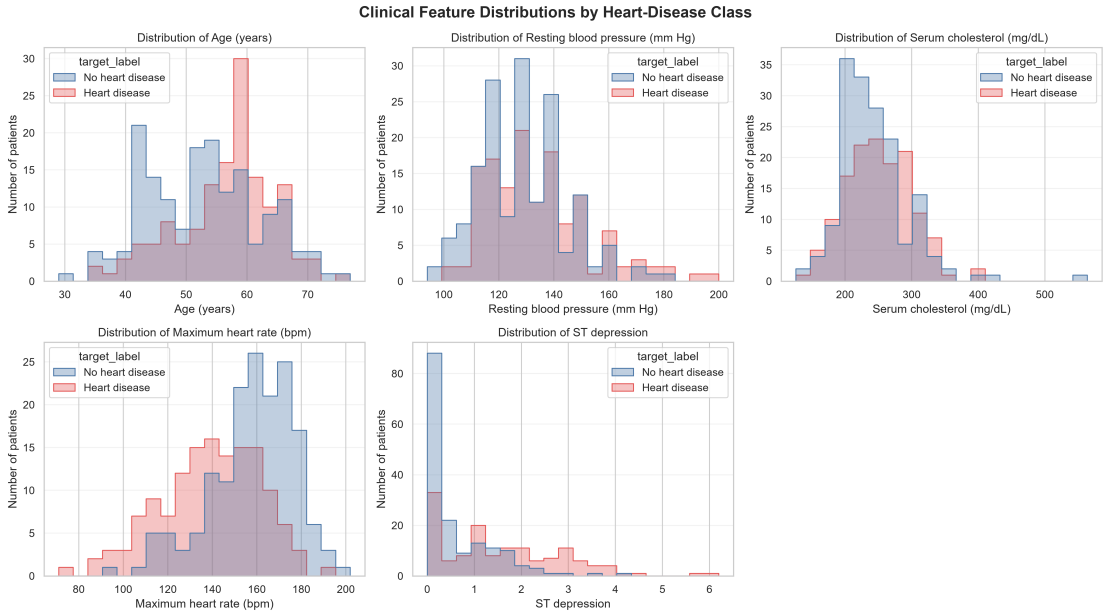


Figure 1: Figure 1. Distributions of the predictor features.

The charts describe this small, historical dataset; they do not establish clinical relationships or generalize results to a current patient population. Histograms are useful here because they reveal each variable's observed range, shape, and potential skew. The heatmap summarizes pairwise linear association; it is descriptive rather than causal, and encoded clinical categories should not automatically be interpreted as continuous measurements. The class-count chart makes the target distribution visible and supports the use of stratification during validation.

4. Model training and evaluation

Both candidates use a leakage-aware scikit-learn pipeline with median imputation and standard scaling. A stratified 80/20 train/test split uses `random_state=42`. Each candidate also receives five-fold stratified cross-validation on the training split. The metrics below are from the project's latest local MLflow runs and held-out test split:

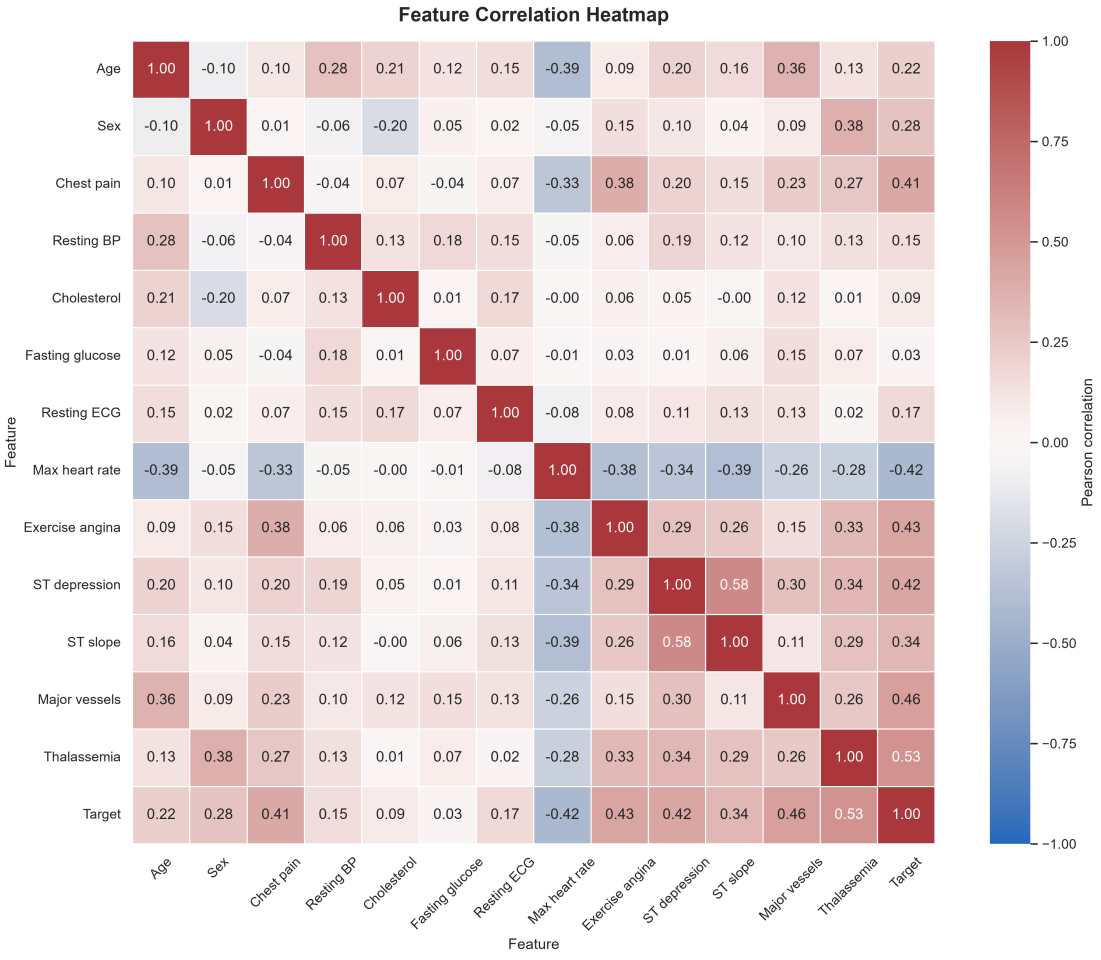


Figure 2: Figure 2. Correlation heatmap for the predictors and target.

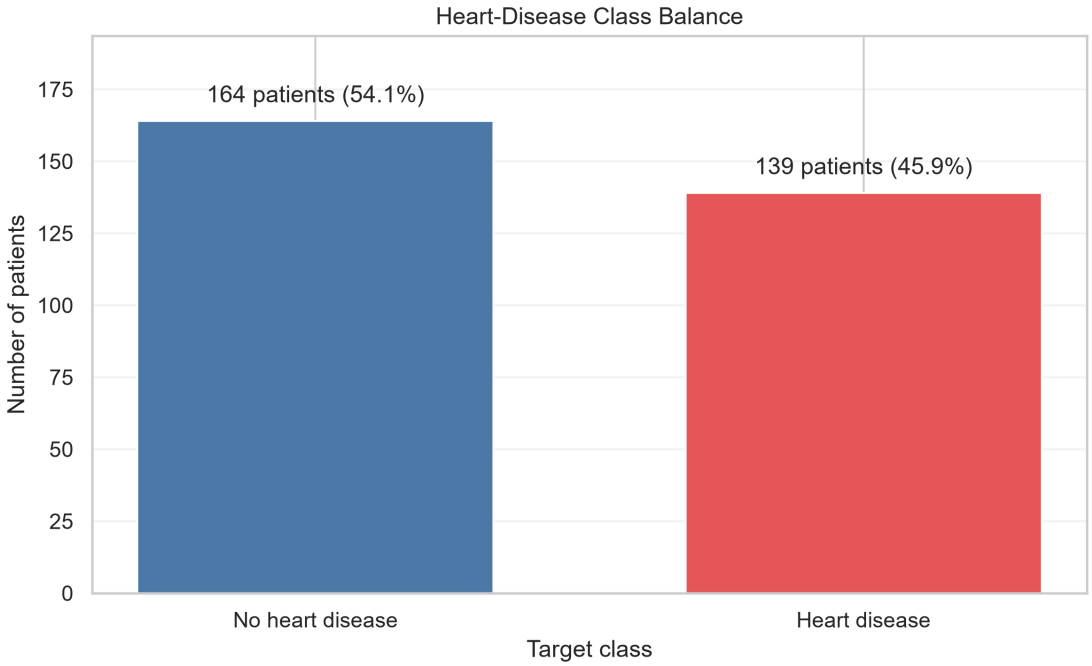


Figure 3: Figure 3. Counts for the two target classes.

Model	CV ROC-AUC (mean +/- SD)	Test accuracy	Test precision	Test recall	Test ROC-AUC
Logistic Regression	0.8960 +/- 0.0148	0.8689	0.8125	0.9286	0.9513
Random Forest	0.8818 +/- 0.0281	0.9016	0.8667	0.9286	0.9491

The training script selects the candidate with the highest held-out test ROC-AUC; therefore Logistic Regression is selected in this run and saved as `models/heart_disease_pipeline.joblib`. It has the slightly higher test ROC-AUC, while Random Forest has higher accuracy and precision. Since the held-out test split is also used for model selection, these test metrics may be optimistic and should not be treated as unbiased final performance estimates. Accuracy is the fraction of correct classifications, precision is the fraction of predicted-positive cases that were positive in the test labels, and recall is the fraction of positive test cases recovered by the model. ROC-AUC measures ranking across thresholds; it does not establish calibration or clinical utility. The difference between the candidates' test ROC-AUC values is small, and this single split is not evidence that one model is consistently superior. The saved training summary is `models/training_summary.json`; it is generated locally and

excluded from Git.

MLflow experiment runs are stored in the local `mlruns/` directory, which is also excluded from Git. The GitHub Actions workflow uploads its own run's model, summary, candidate artifacts, and MLflow data for 14 days.

5. API and privacy-conscious logging

The FastAPI service exposes:

- `GET /health` to confirm the model can be loaded.
- `POST /predict` to return a binary prediction, a class-1 probability, and a plain-language prediction label.

The request schema rejects unknown fields (including `target`), category codes outside those observed in the training data, and continuous values outside the observed dataset ranges. These are dataset constraints, not clinical limits. The model response is not medical advice or a diagnosis.

Application logging records the endpoint, HTTP method/status, and a generic event when a prediction is generated. It intentionally does not log request fields, patient values, prediction class, probability, or caller identity. Uvicorn access logging is disabled in the documented local and container commands to avoid adding client IP addresses. This reduces sensitive information in application logs; it does not make the API suitable for processing real patient data.

The logging behavior is covered by a test that verifies successful request and prediction events appear without feature names or example field values. To inspect logs during local operation, use the server terminal or `docker logs heart-disease-api`. In Kubernetes, use:

```
kubectl logs deployment/heart-disease-api
```

Sanitized output from the successful Kubernetes health and prediction checks is retained in the API log evidence file.

6. Docker and Kubernetes deployment

The Docker image is built from the project Dockerfile after local training creates the model binary. The `.dockerignore` excludes notebooks, data, reports, caches, and unrelated training artifacts from the build context. The image was rebuilt after adding privacy-safe logging and disabling Uvicorn's client-address access logs, then loaded into the Docker Desktop cluster.

The Kubernetes manifests define a Deployment with two replicas and a separate LoadBalancer Service. The local Docker Desktop cluster used for verification is a one-node kind-provisioned cluster backed by containerd. Its node does not automatically see images in the Docker CLI image store; the built image was imported into the node before the rollout. Both pods reached Ready, and the service was verified using a local port-forward:

- `GET /health` returned `{"status":"ok", ...}`.
- A valid `POST /predict` returned HTTP 200 with prediction, probability, and label.

  The request path used for local verification is:

```
Client / Swagger UI
        |
        v
Local kubectl port-forward
        |
        v
Kubernetes Service
        |
        +----> API pod 1 (FastAPI, port 8000)
        |
        +----> API pod 2 (FastAPI, port 8000)
```

  The Service routes to ready API pods; the port-forward makes that Service reachable from the developer machine without publishing a public endpoint. The Swagger prediction response and Docker image screenshots are included in the evidence section. A Kubernetes ready-pod screenshot is useful additional evidence and should be captured if requested by the submission rubric.

7. Continuous integration

The `Heart Disease MLOps CI` GitHub Actions workflow runs on pushes to `main` or `master` and on pull requests. It downloads the dataset, trains both models, lints the code, runs unit tests, and uploads the run outputs. The workflow actions use Node.js 24-compatible releases.

The current privacy-safe logging and API test changes were pushed to `main` in commit `698ec0238e904f99b27d8aac64fcedc5071c2eba`. The corresponding workflow completed successfully:

- Workflow: Heart Disease MLOps CI, run #3
- Commit: `698ec0238e904f99b27d8aac64fcedc5071c2eba`
- Result: successful; the workflow finished with a green status.
- Run duration: approximately 1 minute 11 seconds.

8. Verification and evidence

The local API test suite passed with **8 tests passed**, and scoped Flake8 passed after the privacy-logging change. The GitHub Actions run #3 screenshot shows the green workflow result for the commit containing those changes. The test output included an existing Starlette/httpx deprecation warning; it did not fail the tests.

Screenshots are stored in `screenshots/`:

Evidence	Screenshot
GitHub Actions successful run #3	Run #3
Docker image in Docker Desktop	Docker image
API health check in Swagger	Health check
API prediction response in Swagger	Prediction response

Selected verification screenshots

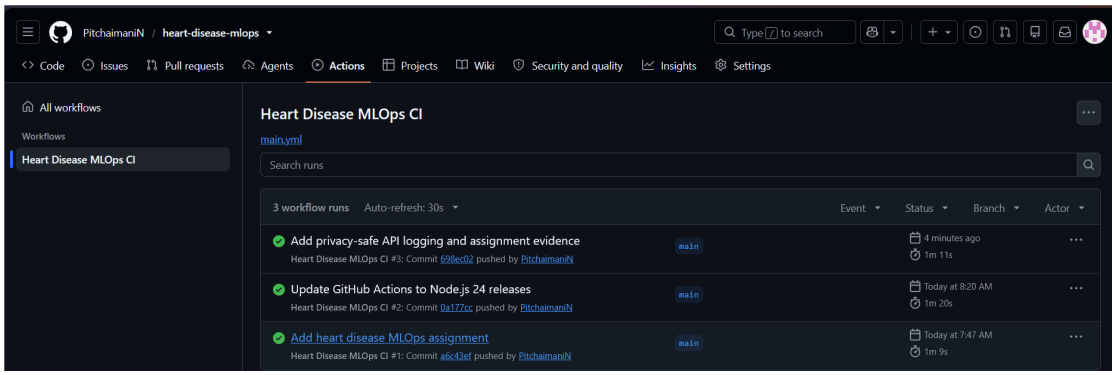


Figure 4: Figure 4. Successful GitHub Actions workflow run #3 for commit 698ec02.

An end-to-end screen-recording video is still to be recorded and added as `evidence/end_to_end_pipeline.mp4`. A screenshot of the Kubernetes ready-pod output should also be added if the assignment requires visual deployment evidence.

9. Limitations

- The Cleveland dataset is small and historical; results may not generalize.
- Test-set-based model selection can bias the reported test metrics upward.
- Dataset ranges and category codes are API input checks, not medical rules.
- Predictions and probabilities are educational outputs, not clinically validated risk estimates.
- No real patient information should be entered into this demonstration API.

10. Reproduction and submission checklist

The repository README contains the step-by-step commands for preparing the data, running EDA and training, starting the API, building the image, and deploying to Kubernetes. In summary, a fresh verification should reproduce these stages in order:

1. Install the packages listed in `requirements.txt` in a Python environment.

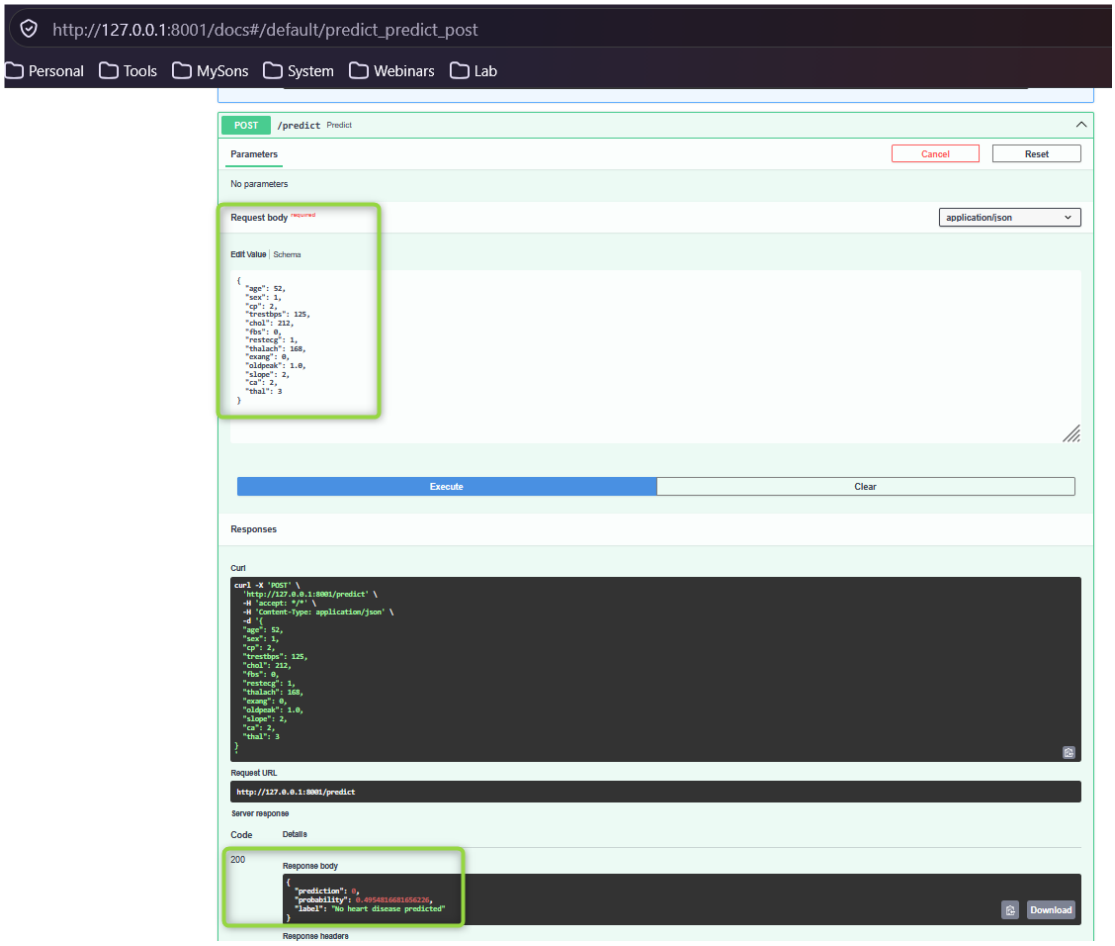


Figure 5: Figure 5. Heart disease API prediction response in Swagger UI.

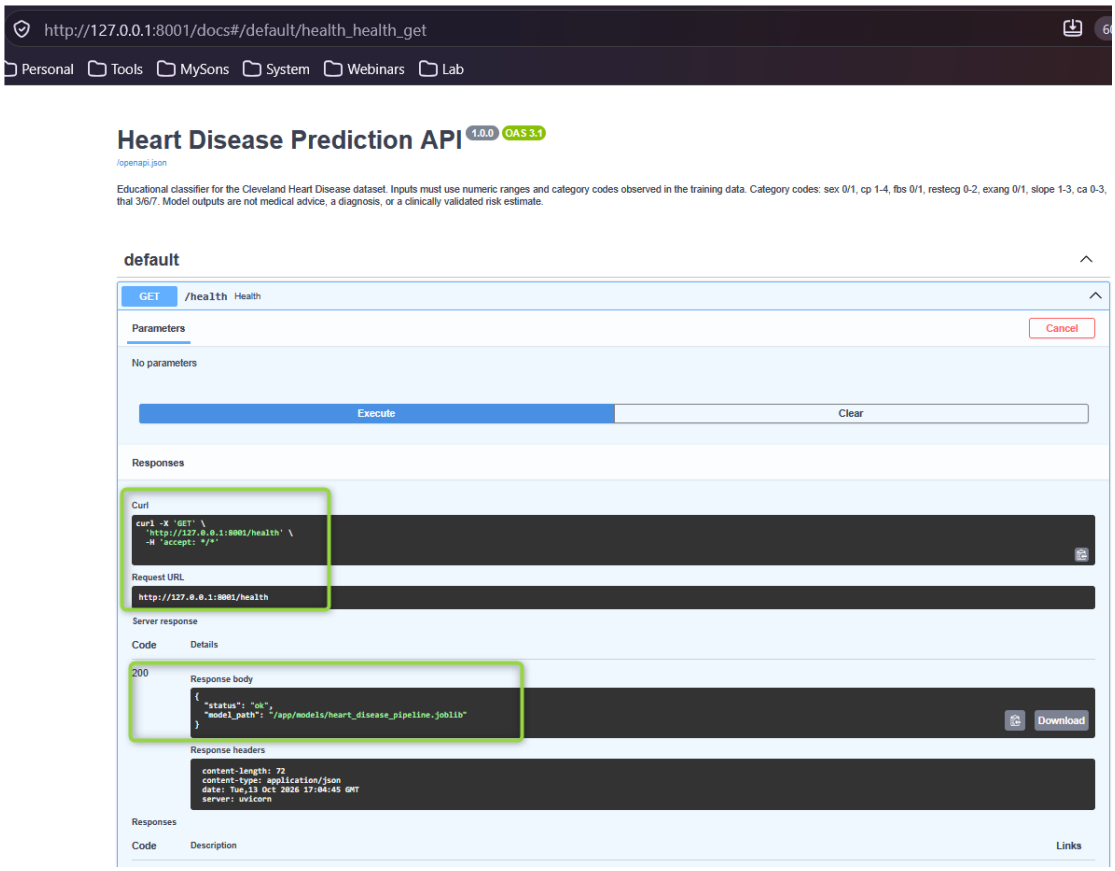


Figure 6: Figure 6. Successful health check in Swagger UI.

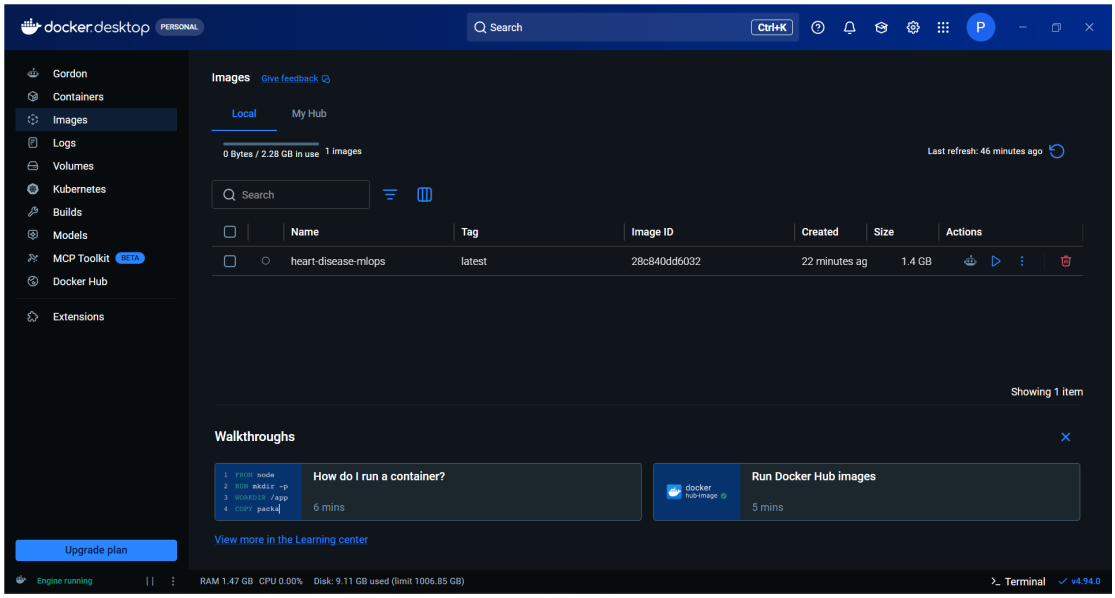


Figure 7: Figure 7. Built API image shown in Docker Desktop.

2. Run `python download.py` to create the processed Cleveland dataset.
3. Run `python train.py` to compare the candidate pipelines and save the final model and MLflow artifacts.
4. Run `pytest -q` and `flake8 .` to check the API/tests and code style.
5. Build and run the Docker image, then apply the Kubernetes Deployment and Service manifests and check that the two replicas become ready.
6. Verify `/health` and `/predict`, inspect privacy-safe application logs, and retain screenshots or a screen recording as evidence.

The checked-in Markdown report is the editable, version-controlled source. The PDF and DOCX exports are formatted submission copies generated from this source. Save the short end-to-end screen recording as `report/evidence/end_to_end_pipeline.mp4`. Before submission, add that video and any additional Kubernetes evidence required by the instructor. Confirm that the report's referenced files are present in the submitted repository.